

머신러닝 기초 I

03 머신러닝을 위한 수학

목차 03 머신러닝을 위한 수학

1. 벡터
2. 행렬
3. 벡터 공간과 차원
4. 선형 변환과 고유값
5. 행렬 분해

목차 03 머신러닝을 위한 수학

6. 미분 기초

7. 편미분

8. 확률 변수와 조건부 확률

9. 통계 분석

1. 벡터



대수적인 방법을 사용하여 **벡터, 행렬, 선형 시스템**을 연구하는 분야

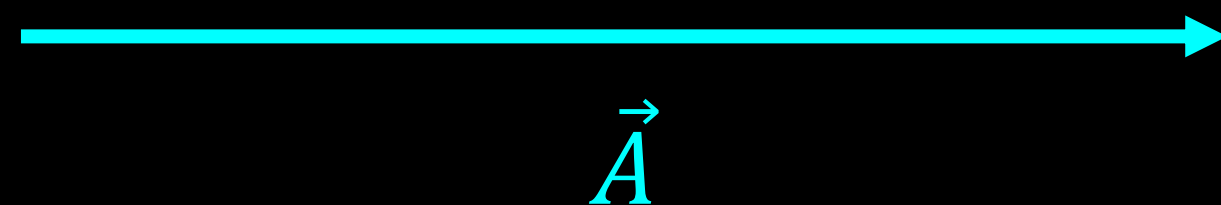
- 행렬과 벡터를 사용해 수많은 데이터를 한 번에 묶어 계산을 쉽게 도움
- 묶인 데이터의 고유 특징을 알려줌으로써 **겉으로 보기 힘든 다양한 특성을 분석**할 수 있음

크기만 있고, 방향은 가지지 않는 정보

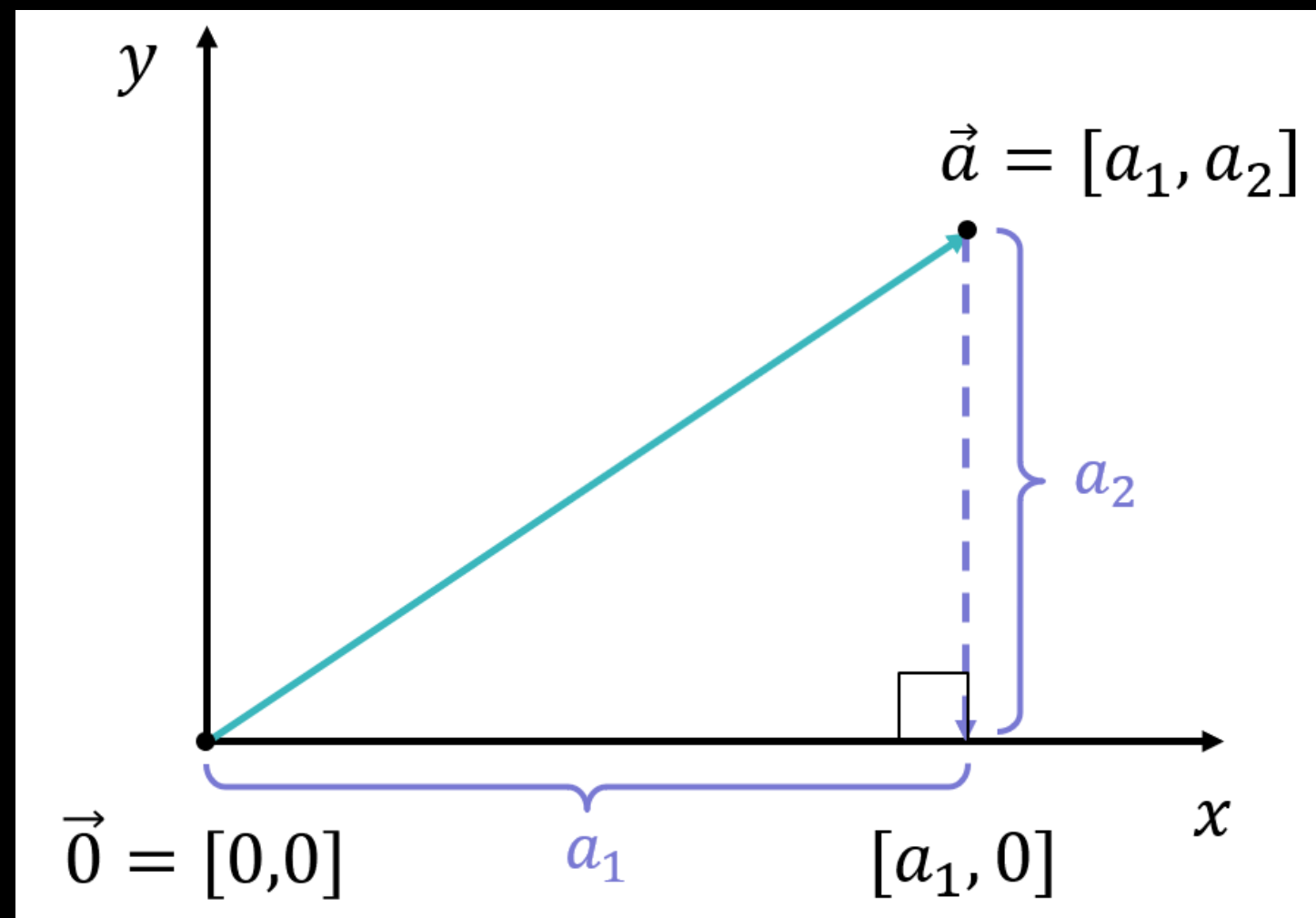
- 위치 정보를 가지고 있지 않음
- 벡터를 곱하여 크기를 조절하거나 방향을 바꿀 수 있음



크기와 방향을 갖는 정보



- 일련의 숫자 리스트
- 공간상에서의 원점으로부터 벡터의 좌표로 함



$$|\vec{a}| = \sqrt{a_1^2 + a_2^2}$$

- 원점에서 벡터의 성분이 가리키고 있는 **좌표를 가리키는 화살표**로 표시
- 벡터 $\vec{a}$ 의 크기 = **직각 삼각형의 빗변의 길이**
→ 피타고라스 정리를 사용해 구할 수 있음



$$|\vec{a}| = \sqrt{\sum_{i=1}^n a_i^2}$$

$$\vec{a} = [a_1, a_2, \dots, a_n]$$

- 2차원 이상의 벡터 크기 계산
- **L2 norm**이라고 불림
- `numpy.linalg.norm` 함수를 이용해 구할 수 있음

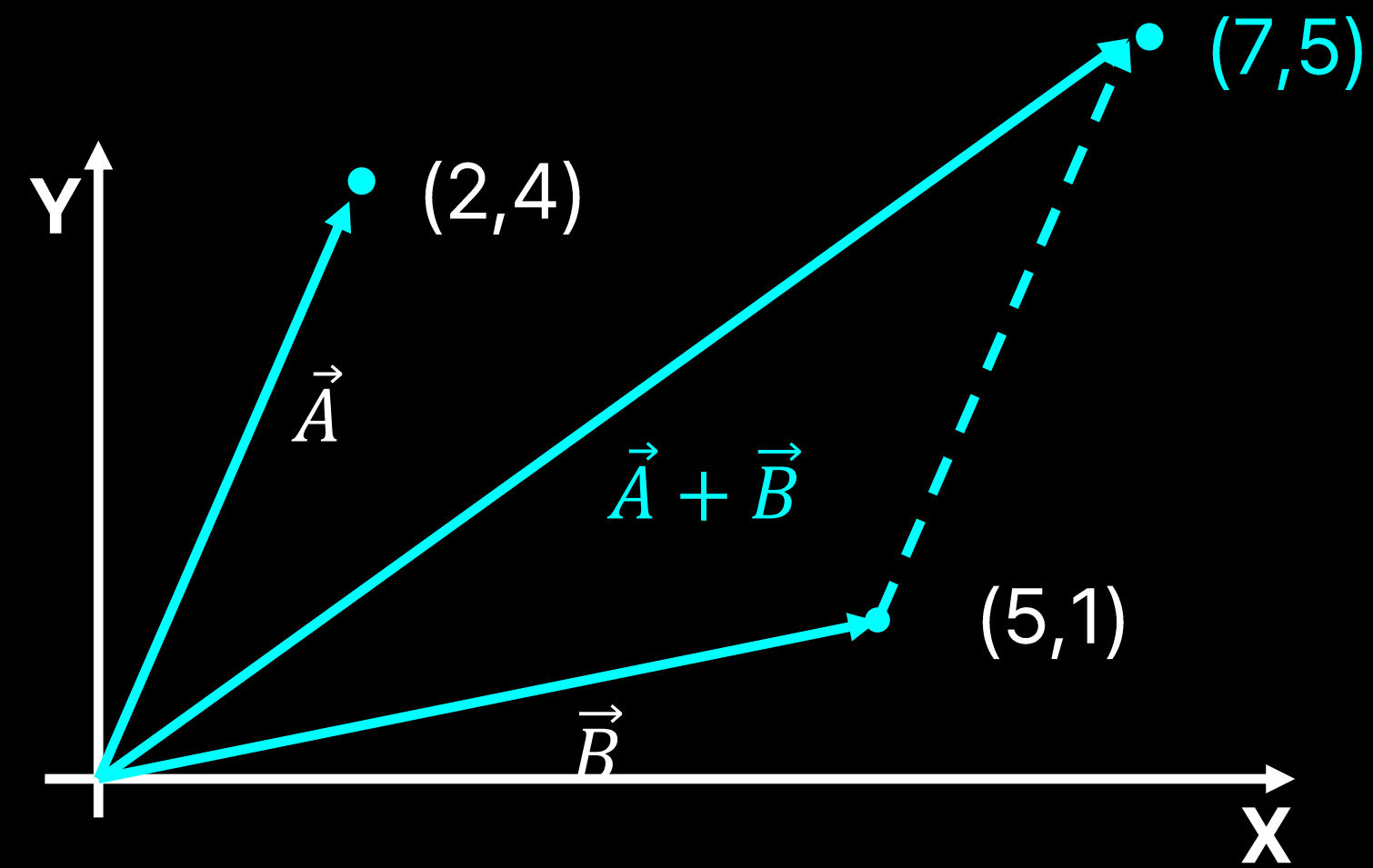


- 벡터에서 $-$ 는 음수를 의미하지 않음
- 방향을 180도 반대로 만들어 주는 역할

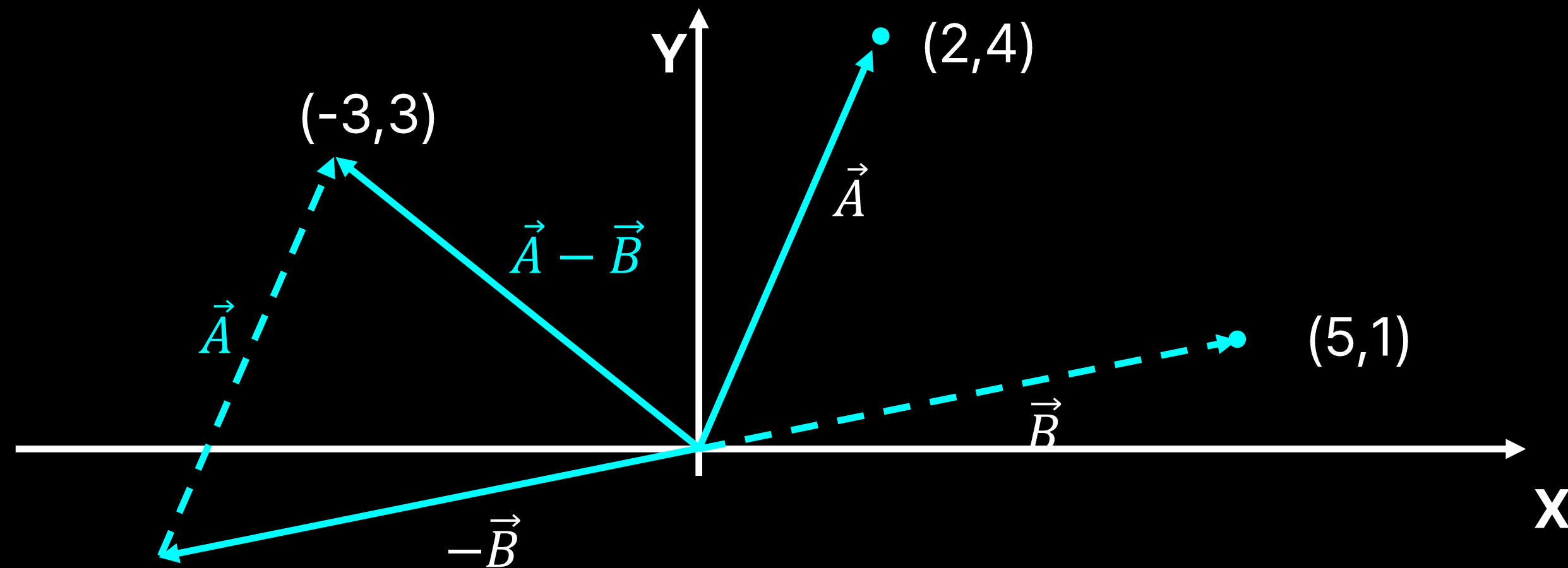


$$\vec{A} + \vec{0} = \vec{0} + \vec{A} = \vec{A}$$

- 더해서 자기 자신이 나오게 하는 벡터
- **영벡터**라고 부름
- 2, 3차원에서는 (0,0) 그리고 (0,0,0)으로 표기



- 화살표를 **평행이동하여 더해**주는 방식
- 각 **원소(element)별로 대응되는 값들의 합**으로 계산
 - 예) $\vec{A} = (2,4), \vec{B} = (5,1)$
 - $\vec{A} + \vec{B} = (2 + 5, 4 + 1) = (7,5)$



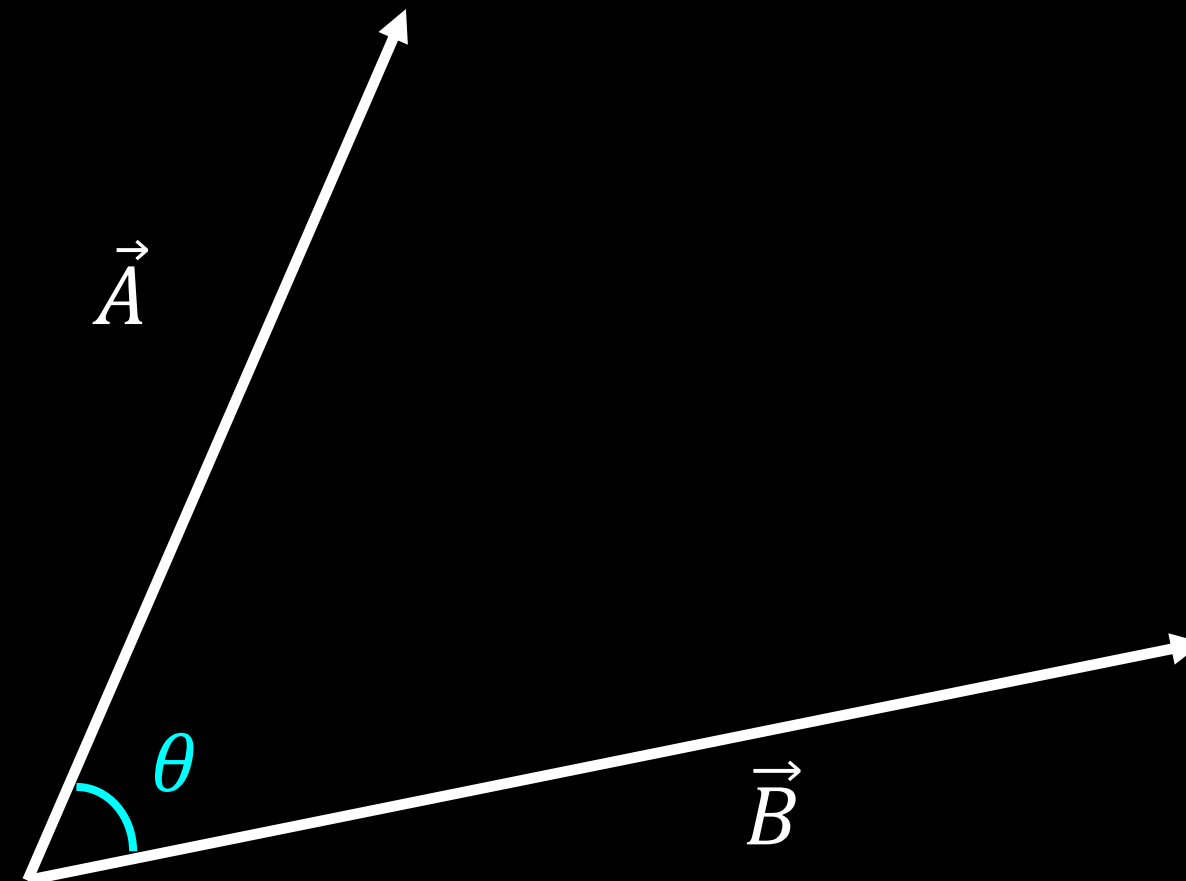
- 덧셈의 응용

- 예) $\vec{A} = (2, 4), \vec{B} = (5, 1)$

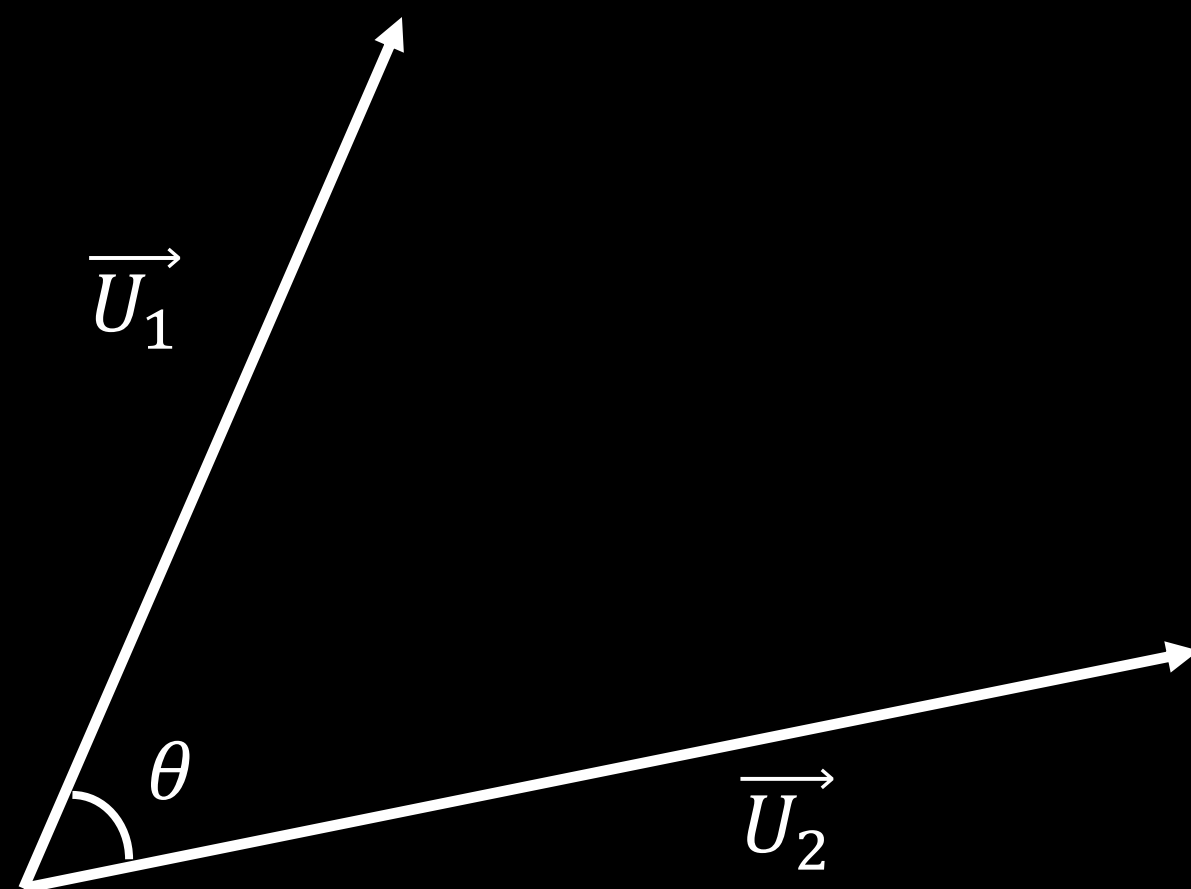
- $\vec{A} - \vec{B} = (2 - 5, 4 - 1) = (-3, 3)$



- 벡터의 내적은 **벡터를 출력하지 않음**
- $\vec{A} \cdot \vec{B}$ 로 표시
- 각 차원 별로 **대응되는 원소(element) 간의 곱의 합**으로 계산
 - 예) $\vec{A} = (2,4), \vec{B} = (5,1)$
 - $\vec{A} \cdot \vec{B} = 2 * 5 + 4 * 1 = 14$



- $\vec{A} \cdot \vec{B} = \|\vec{A}\| \|\vec{B}\| \cos \theta$
- θ : $\vec{A}$ 와 $\vec{B}$ 사이의 사잇각
- $\|\cdot\|$: 괄호 안에 있는 벡터의 크기



- $\vec{u}$ 로 일반적으로 표현
- $\|\vec{u}\| = 1$
- 두 단위 벡터의 내적은 $\cos\theta$



$$\cos\theta = \frac{\vec{A} \cdot \vec{B}}{||\vec{A}|| ||\vec{B}||}$$

- *cosine*값을 내적과 단위 벡터의 활용으로 구할 수 있음
- *cosine*값을 알면 두 벡터 사이의 각도를 역함수를 통해 알 수 있음
- 이를 *cosine similarity*라고 부름



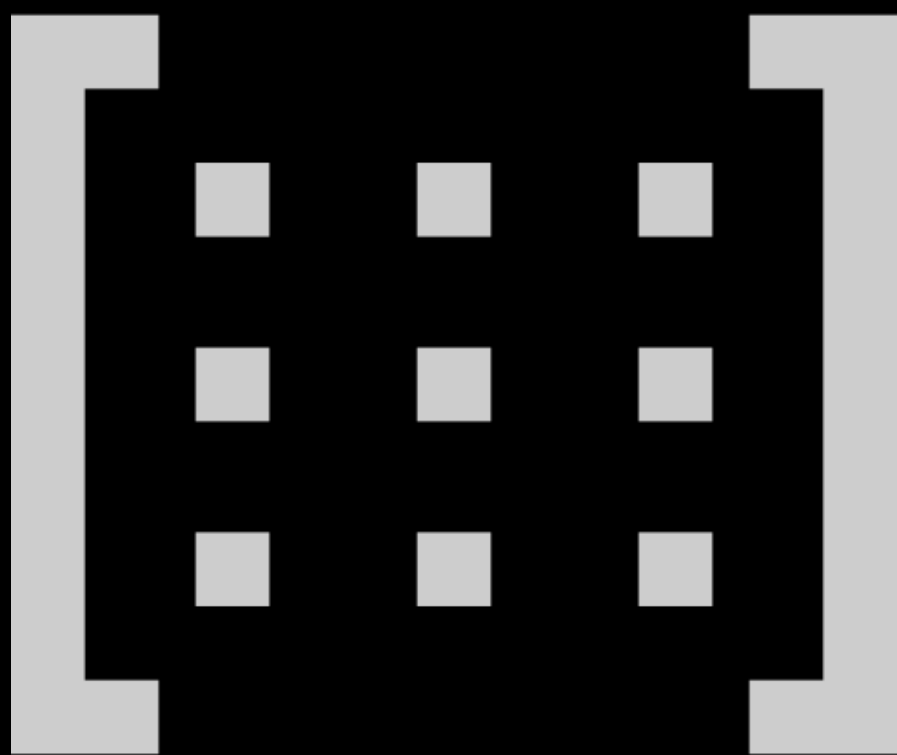
$$\cos\theta = \frac{\vec{A} \cdot \vec{B}}{||\vec{A}|| ||\vec{B}||}$$

- 두 벡터가 직교한다 $\rightarrow \cos\theta = 0$
- 내적이 0일 때, 두 벡터가 0이 아니라면 서로 직교한다.

2. 행렬



수 또는 문자를 괄호 안에 **직사각형 형태**로 배열한 것



행렬(Matrix)



$$x = \begin{bmatrix} 0.2 \\ 1.9 \\ \vdots \\ 5.3 \end{bmatrix}$$

- 머신러닝에선 각각의 샘플들을 위와 같은 **특징 벡터**로 나타냄
- 한 데이터의 집합 $\supset$ 여러 개의 샘플 $\supset$ 수많은 특징
→ **데이터를 간결하게** 표현해야 함



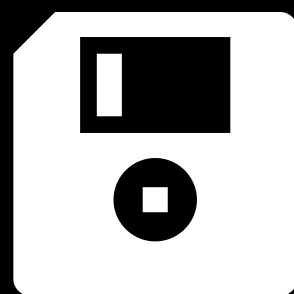
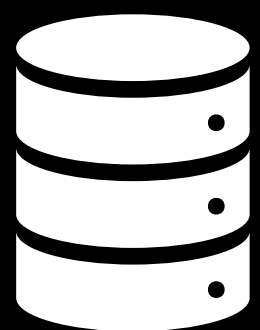
$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$$

- m개의 row, n개의 column으로 이루어짐
- a_{ij} 의 성분으로 구성됨



```
import numpy as np
matrix_A = np.array([[1, 2], [3, 4]])
matrix_B = np.array([[1, 2, 3], [4, 5, 6]])
```

- **numpy 라이브러리**를 이용하여 행렬을 생성할 수 있음
- **다양한 shape**으로 생성 가능



데이터(Data)



철수: [150cm, 52kg, 5세]

선호: [187cm, 69kg, 21세]

...



$$\begin{pmatrix} 150 & 52 & 5 \\ 187 & 69 & 21 \end{pmatrix}$$

- 머신러닝에서 **입력된 데이터는 행렬로 변환**됨

→ 각종 연산을 거쳐 사용되므로 기본적인 행렬 연산에 대한 학습 필요



$$A + O = O + A = A$$

- 더해서 자기 자신이 나오게 하는 행렬
- 행렬의 모든 원소(element)가 0인 행렬을 의미



$$A = \begin{pmatrix} 10 & 2 & 5 \\ 7 & 9 & 1 \end{pmatrix}, B = \begin{pmatrix} -1 & 2 & 3 \\ 7 & -5 & 0 \end{pmatrix}, C = \begin{pmatrix} 5 & -8 \\ 1 & 9 \end{pmatrix}$$

$$A + B = \begin{pmatrix} 10 - 1 & 2 + 2 & 5 + 3 \\ 7 + 7 & 9 - 5 & 1 + 0 \end{pmatrix} = \begin{pmatrix} 9 & 4 & 8 \\ 14 & 4 & 1 \end{pmatrix}$$

$$A + C = ? \text{ 불가능}$$

- 각 원소에 대응되는 것끼리 더하고 빼기
- 덧셈과 뺄셈은 교환법칙이 성립 ($A + B = B + A$)
- 서로 다른 형태의 행렬끼리는 덧셈과 뺄셈을 할 수 없음



```
matrix_A = np.array([[1, 2], [3, 4]])  
matrix_B = np.array ([[5, 6], [7, 8]])  
  
plus = matrix_A + matrix_B  
minus = matrix_A - matrix_B
```

- 행렬의 덧셈과 뺄셈은 +, -로 간단히 구현할 수 있음



$$A = \begin{pmatrix} 10 & 2 & 5 \\ 7 & 9 & 1 \end{pmatrix}, B = \begin{pmatrix} -1 & 2 & 3 \\ 7 & -5 & 0 \end{pmatrix}, C = \begin{pmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$AC = \begin{pmatrix} 10 * 1 + 2 * (-1) + 5 * 1 & 10 * 0 + 2 * 1 + 5 * 0 \\ 7 * 1 + 9 * (-1) + 1 * 1 & 7 * 0 + 9 * 1 + 1 * 0 \end{pmatrix} = \begin{pmatrix} 13 & 2 \\ -1 & 9 \end{pmatrix}$$

$AB = ?$ 불가능

- 첫째 행렬의 열 개수와 둘째 행렬의 행 개수가 동일할 때 행렬곱 가능
- 교환법칙이 성립하지 않음 ($AC \neq CA$)
- 결합 법칙이 성립 ($(AB)C = A(BC)$)



```
import numpy as np
matrix_A = np.array([[1, 2], [3, 4]])
matrix_B = np.array([[5, 6], [7, 8]])
multiple1 = np.dot(matrix_A, matrix_B)
multiple2 = np.matmul(matrix_A, matrix_B)
```

- **dot(a, b)**

- 1차원 배열 : 벡터 내적
- 2차원 배열 : 행렬 곱
- n차원 배열 : 마지막 두 축에 대한 행렬 곱의 합



```
matrix_A = np.array([[1, 2], [3, 4]])  
matrix_B = np.array([[5, 6], [7, 8]])  
  
multiple1 = np.dot(matrix_A, matrix_B)  
multiple2 = np.matmul(matrix_A, matrix_B)
```

- **matmul(a, b)**

- 1차원 배열 : 두 벡터 간의 내적
- 2차원 배열 : 행렬 곱
- n차원 배열 : 행렬 곱 수행, np.dot보다 직관적이고 일관된 방식
- 고차원 배열에 적합



$$A = \begin{pmatrix} 9 & 2 \\ 0 & 1 \end{pmatrix}$$

$$\text{diag}(A) = \begin{pmatrix} 9 \\ 1 \end{pmatrix}$$

$$\text{Trace}(A) = 10$$

- **정사각행렬 : 행과 열의 수가 같은 행렬**
 - n차 정사각행렬로 표현
- 주대각(Diagnoal) : 정사각행렬의 대각선에 위치한 성분
 - $\text{diag}(A)$ 로 표현
- 대각합(Trace) : 정사각행렬의 주대각 element의 합
 - $\text{Trace}(A)$ 로 표현



$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$B = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$

$$\text{Det}(A) = ad - bc$$

$$\text{Det}(B) = a(ei - fh) - b(di - fg) + c(dh - eg)$$

- 정사각행렬에 정의된 값
- 주로 기하학적 해석에서 행렬의 성질
- 행렬식 계산식은 조금씩 다름
- $\text{Det}(A)$ 또는 $\det(A)$ 로 표기



```
matrix_A = np.array([[ -1,  2], [ 4,  0]])  
matrix_B = np.array([[ -1,  2], [ 1, -2]])  
  
det_A = np.linalg.det(matrix_A) # -7.9999999999999998  
det_B = np.linalg.det(matrix_B) # 0.0
```

- **linalg.det(A)**

- 행렬 A에 대한 행렬식을 구해주는 함수



$$I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Identity Matrix, I 로 표기
- 주대각선의 원소가 모두 1이고 나머지가 0인 정사각행렬
- $AI=IA=A$ 성립



```
identity_matrix = np.eye(2)  
identity_matrix
```

```
[[1. 0.]  
 [0. 1.]]
```

- **eye(n)**
 - $n \times n$ 형태의 단위 행렬을 생성해주는 함수



$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

$$\text{Det}(A) = A_{11}A_{22} - A_{12}A_{21}$$

$$A^{-1} = \frac{1}{\text{Det}(A)} = \frac{1}{A_{11}A_{22} - A_{12}A_{21}} \begin{pmatrix} A_{22} & -A_{12} \\ -A_{21} & A_{11} \end{pmatrix}$$

- Inverse Matrix
- 정사각행렬 **A**에 대해 곱해서 단위행렬이 나오게 하는 행렬
- A^{-1} 이라고 표기



$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

$$\text{Det}(A) = A_{11}A_{22} - A_{12}A_{21}$$

$$A^{-1} = \frac{1}{A_{11}A_{22} - A_{12}A_{21}} \begin{pmatrix} A_{22} & -A_{12} \\ -A_{21} & A_{11} \end{pmatrix}$$

- 반드시 정사각행렬에 대해서만 정의
- 어떤 행렬 A 가 역행렬을 가지면, 그 역행렬은 유일함
- 행렬식이 0이라면, A 는 역행렬을 가지지 않음



$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

$$\text{Det}(A) = A_{11}A_{22} - A_{12}A_{21}$$

$$A^{-1} = \frac{1}{A_{11}A_{22} - A_{12}A_{21}} \begin{pmatrix} A_{22} & -A_{12} \\ -A_{21} & A_{11} \end{pmatrix}$$

- $(A^{-1})^{-1} = A$
- $(cA)^{-1} = \frac{1}{c}A^{-1}$
- $(AB)^{-1} = B^{-1}A^{-1}$



```
matrix_A = np.array([[ -1,  2], [ 4,  0]])  
matrix_B = np.array([[ -1,  2], [ 1, -2]])  
inv_A = np.linalg.inv(matrix_A)  
inv_B = np.linalg.inv(matrix_B)      # Error
```

```
[[0.    , 0.25 ],  
 [0.5   , 0.125]]
```

행렬 A의 역행렬 inv_A

- **linalg.inv(A)**

- 행렬 A에 대한 역행렬을 구해주는 함수
- 행렬 A의 행렬식이 0일 경우, Error



$$A = \begin{pmatrix} a & b \\ c & d \\ e & f \end{pmatrix}, \quad A^T = \begin{pmatrix} a & c & e \\ b & d & f \end{pmatrix}$$

- Transpose of Matrix
- **행렬 성분의 행과 열 좌표를 바꾼 행렬**
- A 의 (i, j) 위치에 있는 요소가 A^T 의 (j, i) 위치로 이동



$$A = \begin{pmatrix} a & b \\ c & d \\ e & f \end{pmatrix}, \quad A^T = \begin{pmatrix} a & c & e \\ b & d & f \end{pmatrix}$$

- 대칭 행렬
 - $A = A^T$
- 전치행렬의 전치행렬은 원래 행렬
 - $(A^T)^T = A,$
- 곱셈의 전치
 - $(AB)^T = B^T A^T$



```
matrix_A = np.array([[1, 2], [3, 4]])  
transpose_A = matrix_A.transpose()
```

```
[[1 3]  
 [2 4]]
```

- **.transpose()**
 - 기존 행렬의 전치 행렬을 구해주는 함수

3. 벡터 공간과 차원



어떤 벡터 집합이 있을 때,
그 벡터들의 **선형 결합**으로 구성할 수 있는 벡터들의 집합

$$V_1 : \{ \vec{a}, \vec{b}, \vec{a} + \vec{b}, \dots \}$$

$$V_2 : \{ \vec{c}, \vec{b}, \vec{c} + \vec{b}, \dots \}$$



주어진 벡터들의 **스칼라 곱과 덧셈**으로 구성된 벡터

$$\vec{a} + \vec{b} = [1, -1, 1]^T$$

$$2\vec{a} = [2, 0, 4]^T$$

$$-\vec{b} = [0, 1, 1]^T$$



주어진 벡터들이 서로 독립적

벡터들의 선형 결합이 0이 될 때, 모든 스칼라가 0인 경우에 성립

$$c_1 \vec{a_1} + c_2 \vec{a_2} + \cdots + c_n \vec{a_n} \neq 0, \text{ except for all } c_i = 0$$

<선형 독립 조건>



벡터 공간을 생성하는 최소한의 선형 독립 벡터 집합

$$\vec{b}_1 = [1, 0]^T, \vec{b}_2 = [0, 1]^T$$

- b_1 과 b_2 는 선형 독립이며, 선형 결합을 통해 2차원 상에 존재하는 모든 벡터를 나타낼 수 있음

주어진 벡터들의 모든 선형 결합으로 생성되는 공간

$$\vec{a}, 2\vec{a}, 3\vec{a}, \dots$$

- 벡터 하나에 대해서도 가능



행렬 A 와 곱했을 때, 0벡터를 만드는 모든 벡터들의 집합

- $Ax=0$ 을 만족 시키는 모든 벡터 x
- Null space는 벡터 공간으로, null space에 속하는 선형 결합도 null space에 속함
- 항상 영벡터를 포함



행렬의 열 벡터들이 생성하는 공간

- 벡터 공간이므로, 열 공간의 벡터와 스칼라의 곱은 열 공간에 속함
- 열 공간의 차원은 행렬의 랭크와 동일



차원 : 어떠한 벡터 공간을 이루는 기저의 개수

랭크 : 행렬에서 선형 독립인 행 또는 열 벡터의 최대 수

- 랭크는 $rank(A) = r$ 로 표현
- 랭크의 최솟값은 1
 - 1개의 기저는 항상 존재



$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

1. 벡터 공간

- 행렬 A 의 열 벡터들은 3차원 실수 **벡터 공간 R^3** 에 속함



$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

$$c_1 \begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix} + c_2 \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix} + c_3 \begin{pmatrix} 3 \\ 6 \\ 9 \end{pmatrix}$$

2. 선형 결합

- 모든 벡터들은 **열 벡터들의 선형 결합으로** 구성됨



$$c_1 \begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix} + c_2 \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix} + c_3 \begin{pmatrix} 3 \\ 6 \\ 9 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

3. 선형 독립

- $c_1 = c_2 = c_3 = 0$ 인 경우 외, 식을 만족시키는 c_1, c_2, c_3 값이 있으므로

행렬 A 는 선형 독립이 아님

- $c_1 = 1, c_2 = -2, c_3 = 1$



$$\begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}, \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix}$$

4. 기저

- A 의 열 벡터들이 선형 독립이 아니므로, **A 의 열 벡터 중 선형 독립인 벡터들만 선택**
- 위 두 벡터만 선형 독립으로, A 의 열 벡터 공간을 생성하는 기저



$$Ax = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

5. Null space

- 위 식을 풀면, 행렬의 행들 사이의 선형 종속성으로 인해 하나의 자유 변수가 생김
- **Null space는 1차원 벡터 공간**



$$\text{Column Space of } A = \text{Span} \left\{ \begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}, \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix} \right\}$$

6. Column space

- 행렬 A 의 열 벡터들이 생성하는 공간
- $\mathbb{R}^3$ 의 부분 공간



$$\begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}, \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix}$$

7. 차원

- 기저 벡터의 수와 동일하므로, 행렬 **A**의 열 벡터 공간의 차원은 2



$$\begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}, \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix}$$

8. 랭크

- 행렬에서 선형 독립인 행 또는 열 벡터의 최대의 수
- **행렬 A 의 랭크는 2**



```
import numpy as np
from scipy.linalg import null_space

A = np.array([[2, 1, -1], [4, 2, -2], [6, 3, -3]])

rank_A = np.linalg.matrix_rank(A)
null_space_A = scipy.linalg.null_space(A)
```

```
[[-0.57735027,  0.          ],
 [ 0.57735027,  0.70710678],
 [-0.57735027,  0.70710678]]
```

- **linalg.matrix_rank()**

- 행렬의 rank값을 출력해주는 함수

- **linalg.null_space()**

- 행렬의 null spaces에 해당하는 column 벡터를 출력
- 해당 벡터는 크기가 1로 normalization됨

4. 선형 변환과 고유값



벡터 공간을 다른 벡터 공간으로 사상(mapping)하는 함수

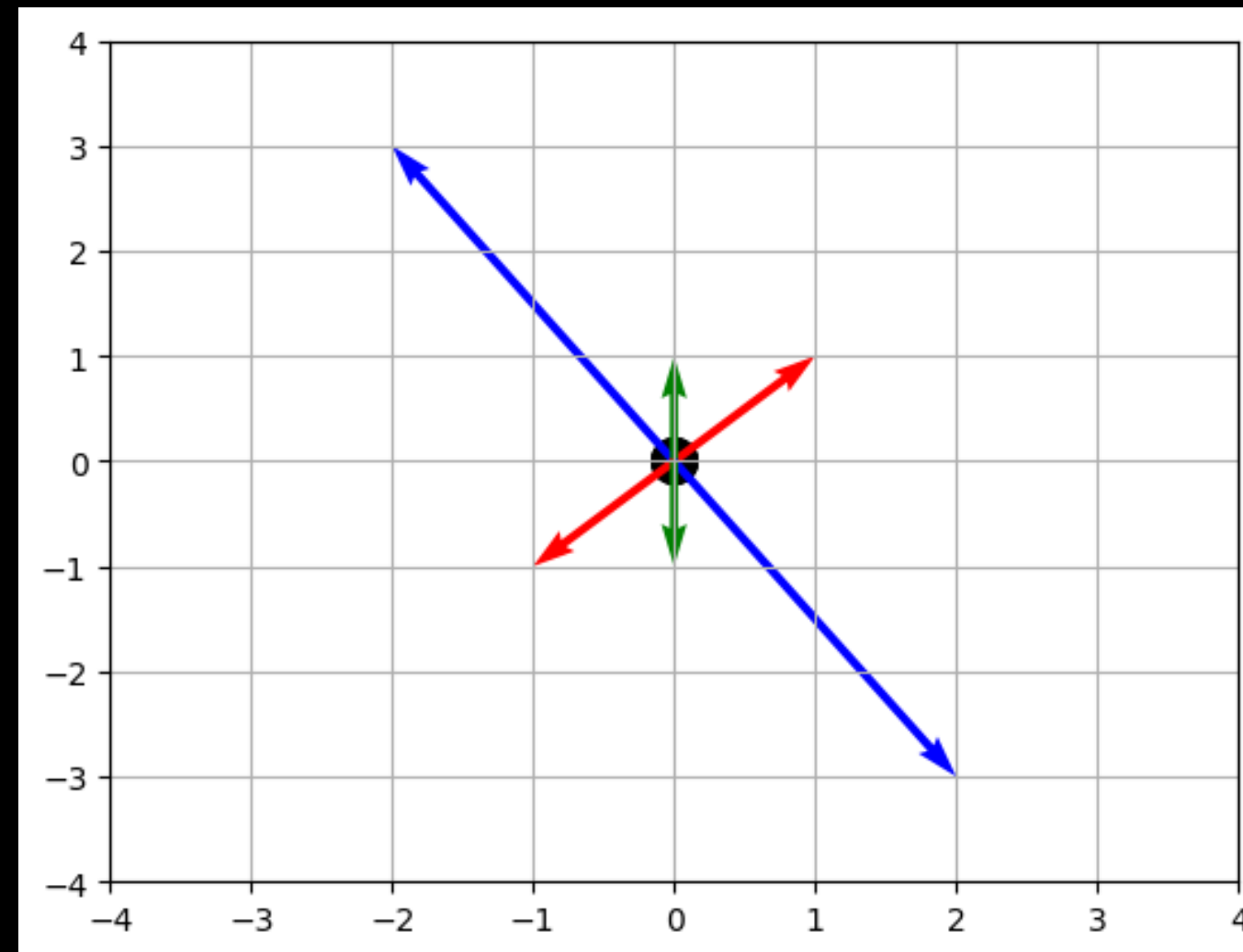
- 덧셈에 대한 보존
 - $L(v + w) = L(v) + L(w)$
- 스칼라 곱에 대한 보존
 - $L(cv) = cL(v)$



$[1, 1]$ 벡터가 있을 때, 이를 **반대 방향으로 바꿔주는 선형 변환 함수 $L_1()$** 이 있다면

$$L_1([1,1]) = [-1, -1]$$

$$L([1,1]) = A[1,1]^T = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$



- 데이터를 변환하거나 모델을 구축할 때 사용
- 3차원 배열로 만들어진 사진 데이터를 다루기 쉬운 1차원 데이터로 변환하는 **다양한 인코딩 과정**으로 사용
- 머신러닝 **모델의 선형 방정식을 푸는 과정**으로도 활용



선형 변환 A 에 의해 **방향이 변하지 않고 크기만 변하는 벡터**

- $Av = \lambda v$
- v 는 고유 벡터를 의미하며, 일반적으로 column의 크기가 1인 column 벡터
- λ 는 고유벡터의 크기를 변화시키는 스칼라 값
 - **고유값**으로 불림



```
A = np.array([[2, 1, -1], [4, 2, -2], [6, 3, -3]])  
  
A_eig_val, A_eig_vec = np.linalg.eig(A)
```

```
[ 1.00000000e+00,  1.17966505e-15, -3.17033140e-16]  
[[-0.26726124  0.19897406  0.09781532],  
 [-0.53452248  0.46481354 -0.79469999],  
 [-0.80178373  0.86276167 -0.59906935]]
```

A_eig_val, A_eig_vec

- **linalg.eig(A)**

- 행렬 A의 고유값과 고유벡터를 출력해주는 함수
- A_eig_val = 각 고유값을 포함하는 1차원 배열
- A_eig_vec = 각 열이 고유벡터를 나타내는 2차원 배열
- 이때 출력되는 고유벡터는 크기가 1이 되도록 normalization됨
- 행이 아닌 **열을 고유벡터로 가진다**는 점에 주의



- 데이터가 담긴 행렬을 단순히 **테이블로 보면**,
→ 각 데이터가 어떤 특성이 있는지, 어떤 패턴으로 이루어져 있는지 확인 X
- 행렬의 **고유값과 고유 벡터를 분석하면**,
→ 행렬의 핵심 특징을 더욱 쉽게 알 수 있음

5. 행렬 분해



주어진 행렬을 두 개 이상의 특수한 형태의 행렬로 나누는 과정

- 행렬의 속성을 분석하거나 계산을 용이하게 함
- 다양한 알고리즘과 데이터 처리 작업에서 필수적인 도구



- 2개의 고유값을 갖는 A

$$Av_1 = \lambda_1 v_1$$

$$Av_2 = \lambda_2 v_2$$

- 위의 두 방정식을 합치면

$$A[v_1, v_2] = [\lambda_1 v_1, \lambda_2 v_2]$$

$$A[v_1, v_2] = [v_1, v_2] \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

$$AV = V\Lambda$$

- 행렬 A 에 대해서 정리하면

$$AV = V\Lambda V^{-1}$$



$$AV = V\Lambda V^{-1}$$

행렬 A 를 고유 벡터들의 행렬인 V 와 고유값을 대각 성분으로 가지는 Λ 대각 행렬을 사용해

“고유값을 분해했다” 표현



- **n** 제공 계산량의 완화
 - 머신러닝 상에서 행렬 계산량을 대폭 줄여줌
 - 계산 시간 줄어듦
- 사용 분야 예시) 차원 축소, 데이터 압축, 추천 시스템 등 다양한 응용 분야



```
A_l_v, A_diag, A_r_v = np.linalg.svd(A)
```

```
[[-0.26726124,  0.95618289,  0.11952286],  
 [-0.53452248, -0.04390192, -0.84401323],  
 [-0.80178373, -0.28945968,  0.52283453]]
```

```
[9.16515139e+00, 1.14663341e-15, 2.18952885e-47]
```

```
[[-0.81649658, -0.40824829,  0.40824829],  
 [-0.57735027,  0.57735027, -0.57735027],  
 [ 0.          ,  0.70710678,  0.70710678]]
```

A_l_v, A_diag, A_r_v

- **linalg.svd(A)**

- 행렬의 특이값 분해를 수행하는데 사용
- A_l_v = A의 좌측 특이 벡터를 열로 가지는 행렬
- A_diag = A의 특이값을 포함하는 1차원 배열
- A_r_v = A의 우측 특이 벡터를 행으로 가지는 행렬
- 데이터의 주요 패턴 추출 및 대규모 데이터셋 효율적 처리

6. 미분 기초



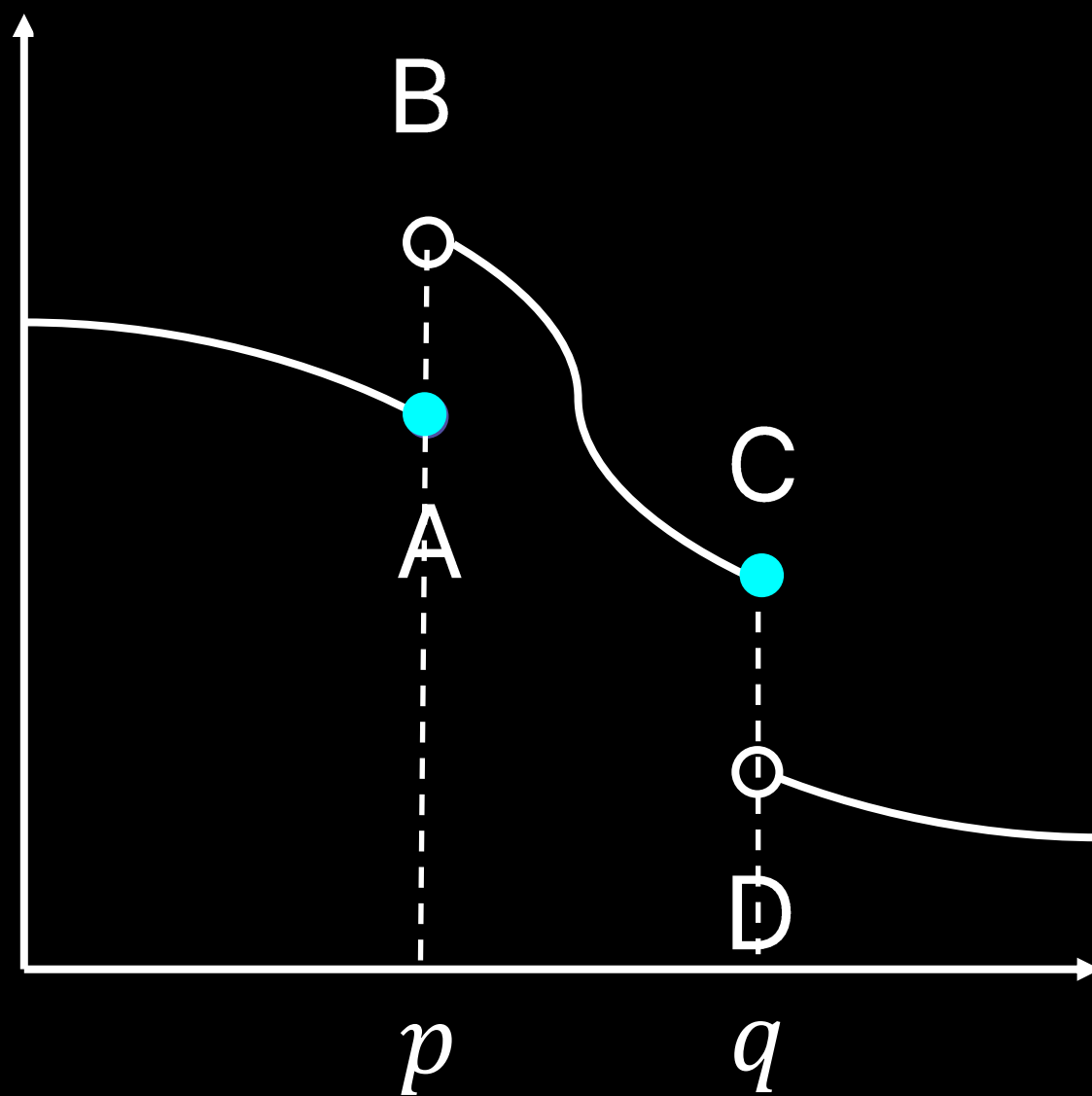
입력값의 순간 변화량에 따른 출력값의 변화량

- 물리적으로 얼마나 빠르게 변화하느냐에 대한 척도
- 함수의 그래프에서 접선의 기울기



- 최댓값 또는 최솟값을 구하는 최적화 방식에 사용
- 예측의 정확도 향상을 위해 파라미터를 조정

→ 정확도를 최대로 만들거나 예측치와의 오차를 최소로 하는 문제 해결 필요



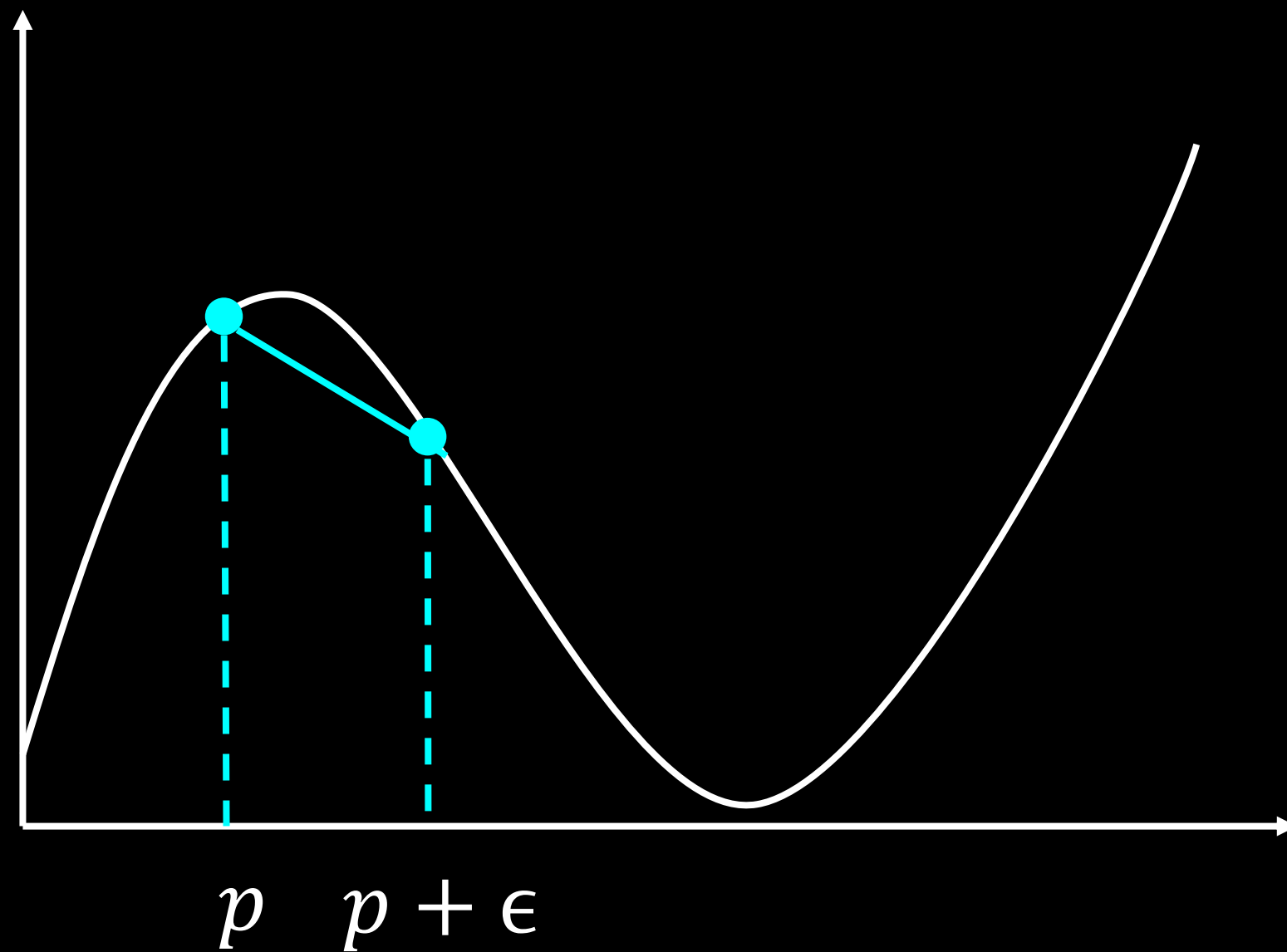
좌극한: $\lim_{x \rightarrow p_-} f(x) = A$

우극한: $\lim_{x \rightarrow p_+} f(x) = B$

좌극한: $\lim_{x \rightarrow q_-} f(x) = C$

우극한: $\lim_{x \rightarrow q_+} f(x) = D$

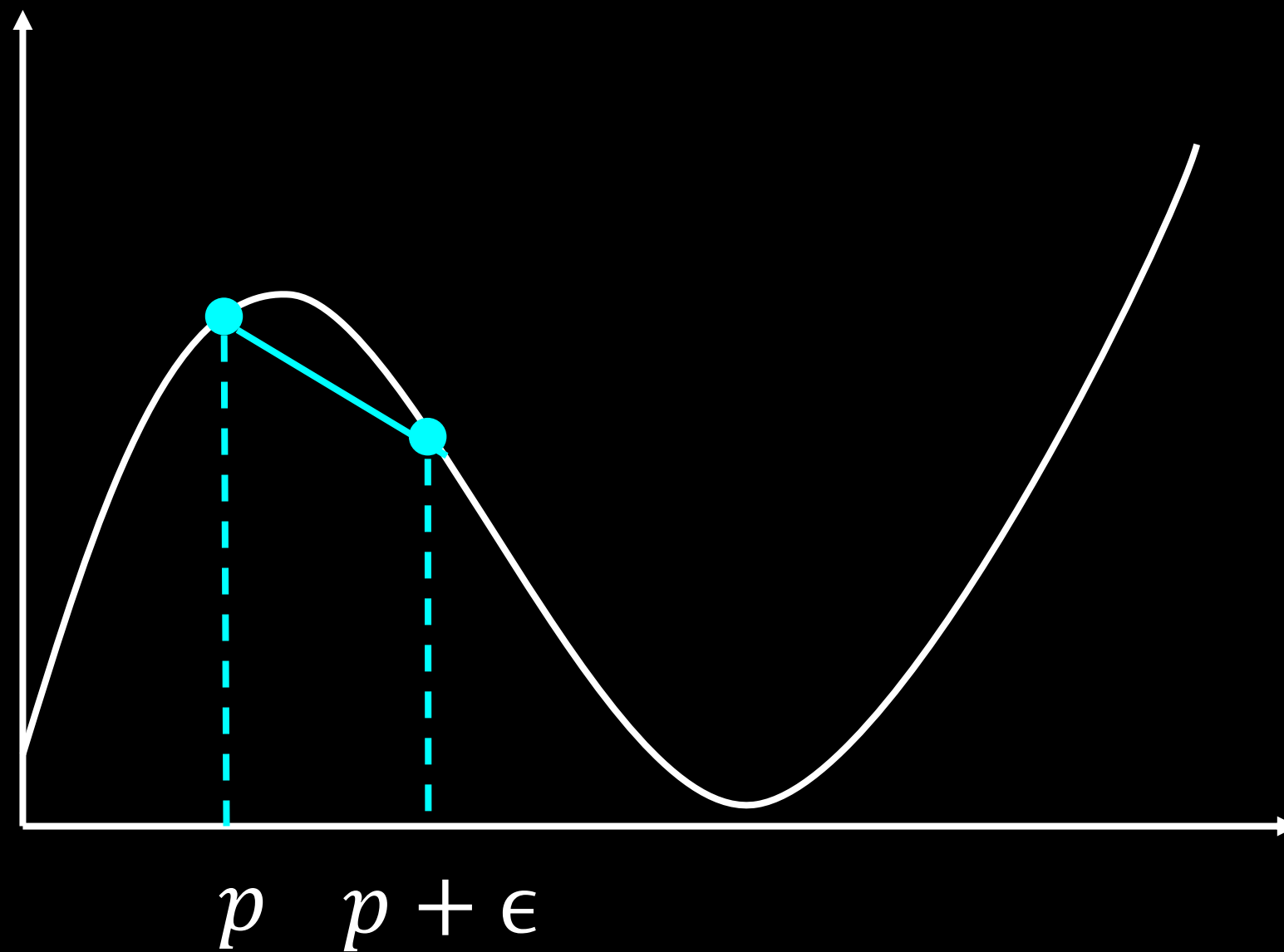
- 좌극한 : 왼쪽에서 오른쪽으로 목표를 이동했을 때, 수렴될 것으로 기대되는 극한값
- 우극한 : 오른쪽에서 왼쪽으로 목표를 이동했을 때, 수렴될 것으로 기대되는 극한값



기울기

$$\frac{f(p + \epsilon) - f(p)}{(p + \epsilon) - p} = \frac{f(p + \epsilon) - f(p)}{\epsilon}$$

- $x = p$ 에서의 접선의 기울기는 $f'(p)$ 로 표기
- $x = p + \epsilon$ ($\epsilon > 0$) 점과의 기울기를 먼저 살펴봄



접선의 기울기

$$f'(p) = \lim_{\epsilon \rightarrow 0} \frac{f(p + \epsilon) - f(p)}{\epsilon}$$

- 기울기에서 epsilon을 0으로 극한을 보냄.
- $\lim_{\epsilon \rightarrow 0} \frac{f(p + \epsilon) - f(p)}{\epsilon}$ 이 위 그림에서의 **접선의 기울기, 미분계수**라고 부름.



- $f(x) = c \rightarrow f'(x) = 0$ (c 는 상수)
- $f(x) = x^n \rightarrow f'(x) = nx^{n-1}$
- $f(x) = e^x \rightarrow f'(x) = e^x$
- $f(x) = \sin(x) \rightarrow f'(x) = \cos(x)$
- $f(x) = \ln(x) \rightarrow f'(x) = \frac{1}{x}$
- $cf(x) \rightarrow cf'(x)$
- $f(x) + g(x) \rightarrow f'(x) + g'(x)$
- $f(x)g(x) \rightarrow f'(x)g(x) + f(x)g'(x)$



함수 안의 변수로 또 다른 함수로 입력되는 함수

- $y = f(z), z = g(x)$ 일 때, $y = f(g(x))$ 로 표현



$$f(x) = g(h(x)) \rightarrow f'(x) = g'(h(x))h'(x)$$

1. $f(x) = g(h(x)) \rightarrow f'(x) = g'(h(x))h'(x)$

2. 먼저, $g(h(x))$ 를 $h(x)$ 에 대해 미분

3. 이후, $h(x)$ 를 x 에 대해 미분

- 딥러닝 모델 학습시 사용되는 **오류 역전파 알고리즘** 또한 연쇄법칙 기반으로 작동



```
import sympy
x = sympy.symbols('x')
sym_f = sympy.expand((x-1)**2)

sym_d_f_x = sympy.diff(sym_f,x)
```

$2*x - 2$

sym_d_f_x

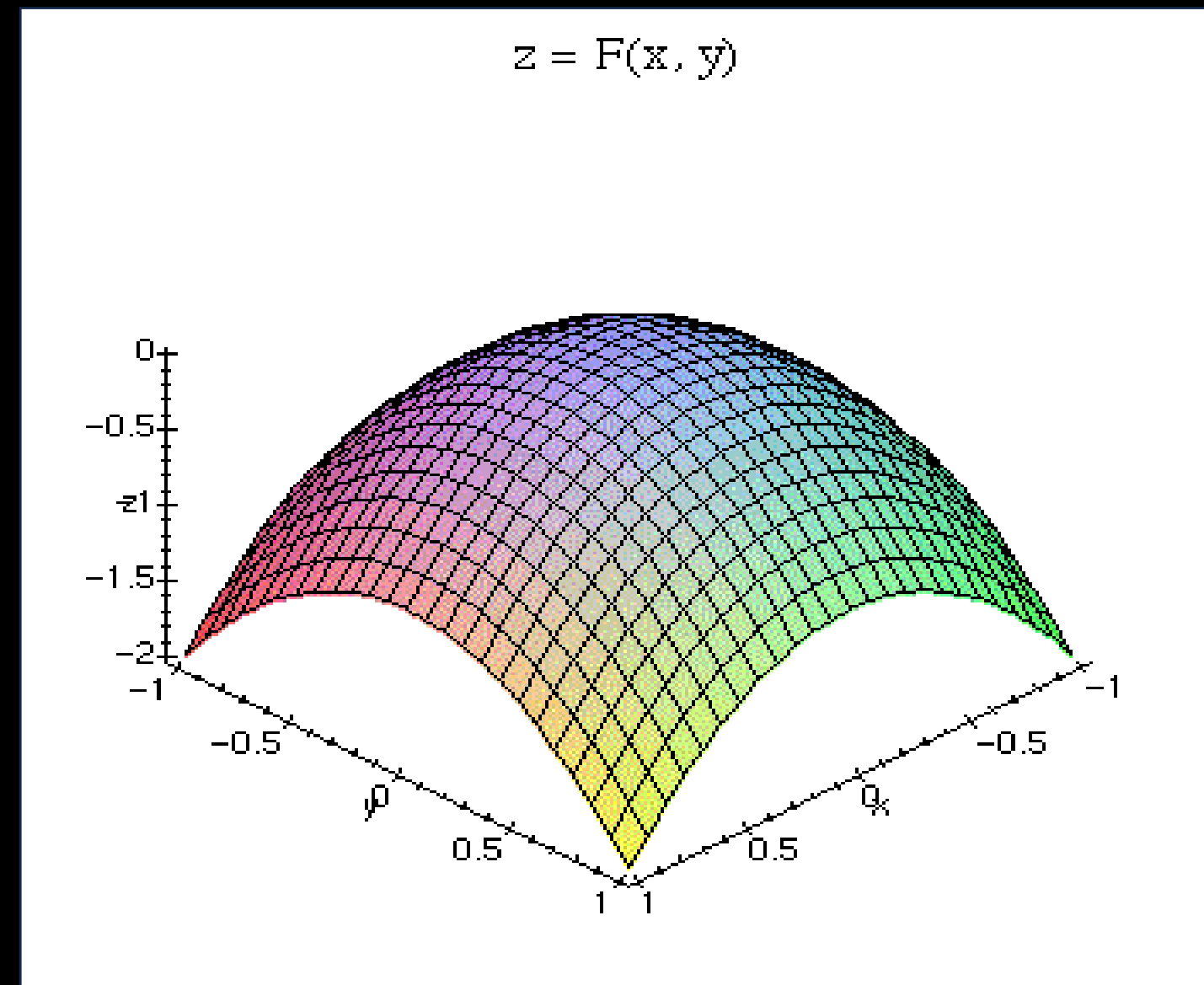
- **symbols()**
 - 변수를 생성하는 함수
- **expand()**
 - 수학적 표현식을 확장하는데 사용하는 함수
- **diff()**
 - 미분을 수행하는 함수

7. 편미분



한 개의 변수가 아닌 **다변수 함수**의 미분

- 다변수 함수 $f(x, y, z, \dots)$
- **다변수 함수의 변수들 간의 미분은 상수 취급**



- $z = f(x, y)$ 는 x, y, z 축의 3차원 위의 도형을 나타냄 (곡면)
- $(x, y) = (0, 0)$ 에서 아래 **곡면에 접하는 평면**을 구하고자 할 때,
 - 각 x 와 y 에 대한 편미분을 이용해 구함
 - **접선의 확장 접평면**



```
import sympy
x, y = sympy.symbols('x y')
sym_f = sympy.expand((x-1-y)**2)

sym_d_f_x = sympy.diff(sym_f, x)
sym_d_f_y = sympy.diff(sym_f, y)
```

```
2*x - 2*y - 2    # sym_d_f_x
-2*x + 2*y + 2    # sym_d_f_y
```

- 미분 코딩과 동일한 방식으로 진행
- symbols()에 x 이외에 추가로 사용할 변수를 정의
- diff()에 함수와 함께 해당 변수 전달시, 편미분 결과를 얻을 수 있음



어떤 목적 함수의 출력값을 최대 혹은 최소화하는 파라미터를 찾는 문제

- 효율적인 자원 할당, 비용 절감, 성능 향상에 사용

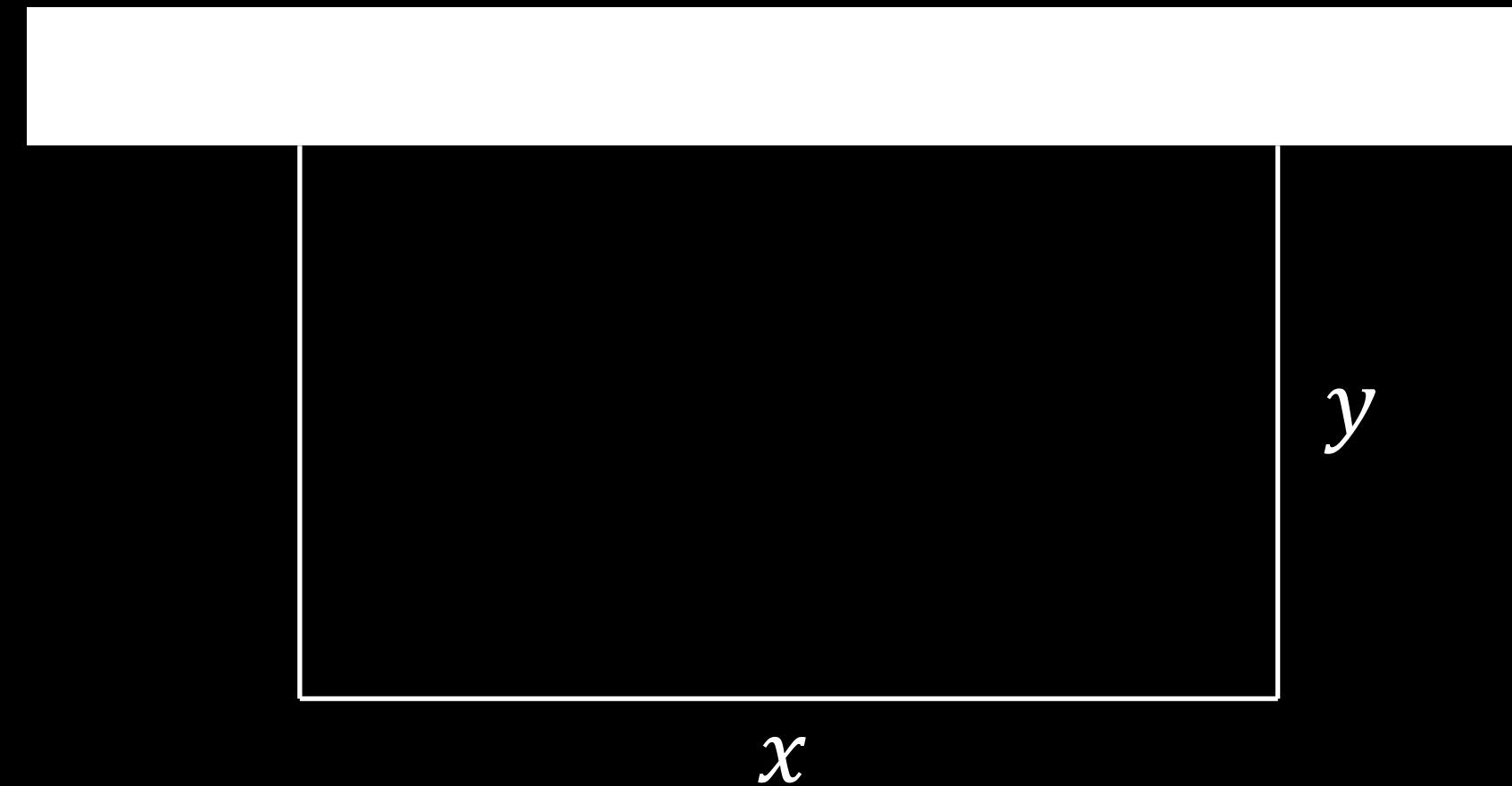
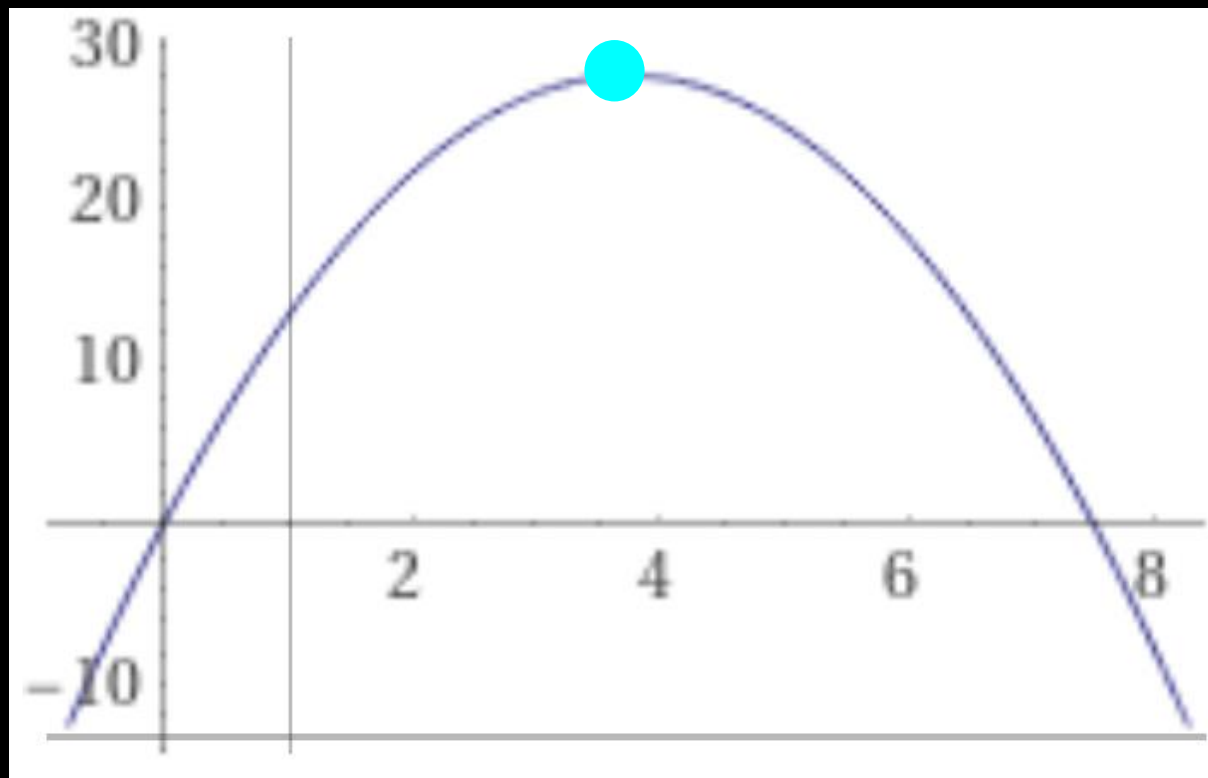


- 대표 최적화 수학 문제
 - 위 그림처럼 직사각형 모양의 울타리를 벽에 치고자 한다. 울타리의 길이가 15m일 때, 창고의 넓이를 최대로 하려면 창고의 가로, 세로의 길이를 어떻게 해야하는가?



최적화 문제 예시

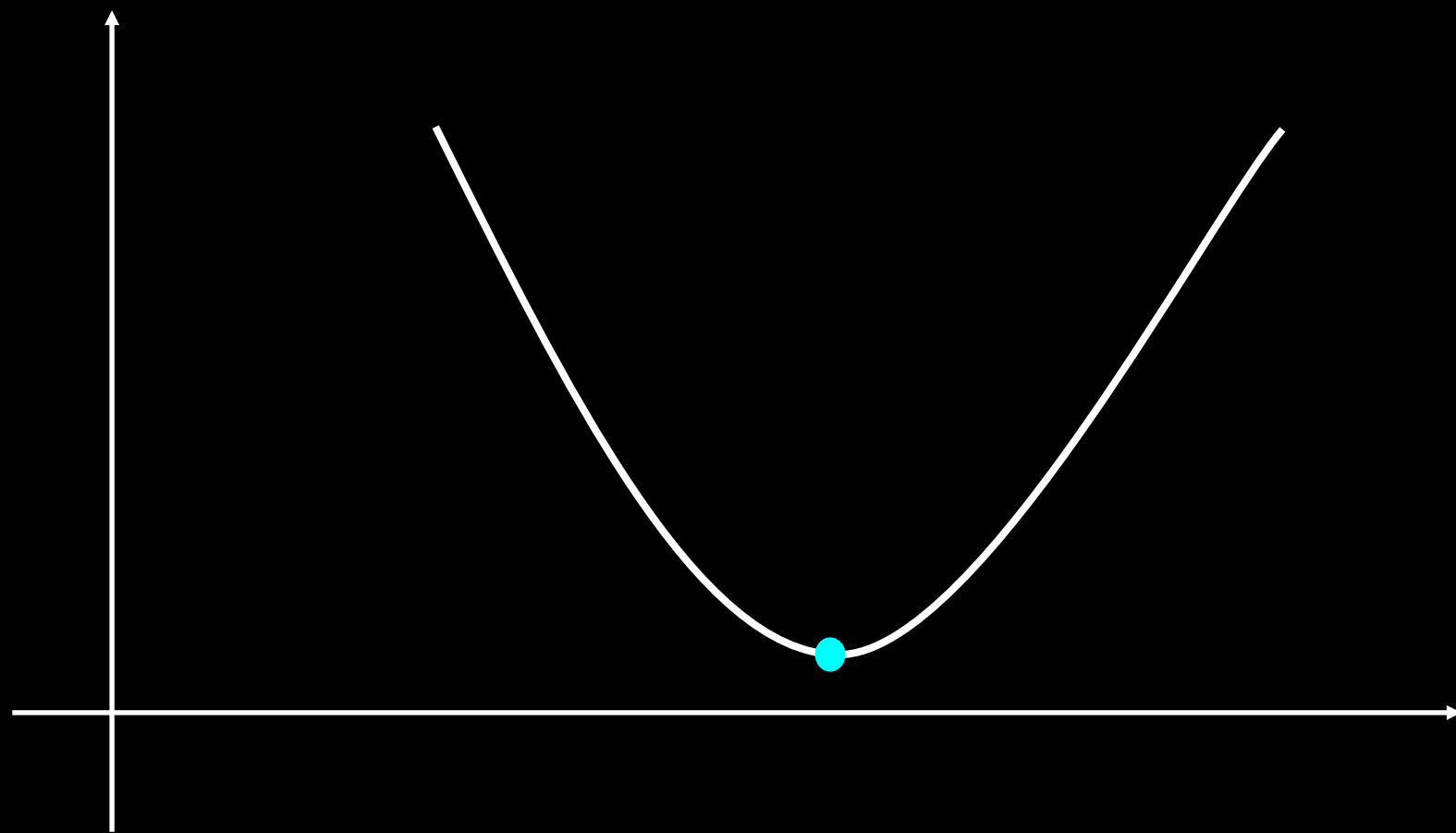
$$f(y) = y(15 - 2y)$$



- 조건: $x+2y=15, x \geq 0, y \geq 0$
- 목적 함수: $xy \leftarrow$ 최대화
- 문제 변형: $\max_y y(15-2y)$

접선의 기울기

경사하강법으로 불리며, 목적 함수의 최소화가 목표일 때 사용



*목적 함수($f(x)$)의 최대화가 목표일 때는 $-f(x)$ 로 치환해서 항상 최소화 문제로 바꿀 수 있음.



```
import sympy
x, y = sympy.symbols('x y')
sym_f = sympy.expand((x-1-y)**2)

sym_d_f_x = sympy.diff(sym_f,x)
sym_d_f_y = sympy.diff(sym_f,y)

def gradient(x,y):
    return 2*x - 2*y - 2, -2*x + 2*y + 2

gradient_point = [1], [2]
gradient_f_x, gradient_f_y = gradient(*gradient_point[0],*gradient_point[1])
```

```
# gradient_f_x, gradient_f_y
(-4, 4)
```

- 기울기를 계산하는 함수를 정의한 뒤, 특정 지점에 대한 gradient를 계산
- $\nabla f(x, y) = (2x - 2y - 2, -2x + 2y + 2)$

8. 확률 변수와 조건부 확률

(그 사건이 일어날 수 있는 경우의 수) / (총 경우의 수)

- 확률 변수 = 특정 사건들을 실수로 대응시킨 것
 - $X \in \{1, 2, 3, \dots\}$ 으로 표현
- 확률 표현
 - $P(X = 1)$, 확률 변수 X 가 1일 확률



```
A = [1, 2, 3, 4]
B = [2, 4, 6, 8, 10, 12]
U = set(A+B)

prob_A = len(A)/len(U)
prob_B = len(B)/len(U)
```

- 간단한 리스트 연산으로 확률을 표현할 수 있음
- 사건 A와 B 정의, 전체 경우의 수 정의
- 나누기 연산을 통해 확률 계산



이산 변수

- 셀 수 있는 수로 이루어진 변수
- 책 권수, 공 개수

연속 변수

- 셀 수 없는 단위를 가지는 수
- 연속적인 수
- 키, 몸무게



확률 변수에 따라 각 확률값을 대응시킨 분포

- 이산 확률 분포
 - 확률 변수가 이산 확률 변수인 경우
 - 이를 나타내는 함수는 확률 질량 함수
- 연속 확률 분포
 - 확률 변수가 연속 확률 변수인 경우
 - 이를 나타내는 함수는 확률 밀도 함수, 누적 분포 함수



어떤 사건이 발생하거나 발생하지 않은 것에 대한 확률 분포

- $P(X = 0) = p, P(X = 1) = 1 - p, 0 \leq p \leq 1$
- 시행이 성공 또는 실패로만 구분되는 경우
 - 예시) 동전을 던졌을 때 앞면 또는 뒷면이 나올 확률



베르누이 분포를 여러 번 했을 때, 그 사건이 몇 번 발생한 것에 대한 확률 분포

- $P(X = x) = \binom{n}{k} p^k (1 - p)^{n-k}, 0 \leq p \leq 1$
- 각 시행은 베르누이 분포와 동일하게 두 가지 결과만 가질 수 있으며, 각 시행의 성공의 확률 p 가 동일해야 함
- n 은 시행 횟수, k 는 성공 횟수



모든 확률 변수에 대해서 **똑같은 확률값**을 가지는 확률 분포

- $P(X = x) = \frac{1}{b-a+1}$
- 이산적인 값들이 균일하게 나타날 때 사용
 - 예시) 주사위를 던질 때 나올 수 있는 각 숫자는 1~6까지의 값을 가지며, 각 값이 나올 확률은 $\frac{1}{6}$ 로 동일



```
import numpy as np
n, p = 1, 0.5

bern_sample = np.random.binomial(n, p, 20)
print("베르누이 분포 샘플: {}".format(bern_sample))
```

베르누이 분포 샘플: [0 0 1 1 0 1 0 1 0 1 1 0 1 1 1 0 0 1 0 1]

- 베르누이 분포
- 베르누이는 시행 1번 $p=0.5$ 로 설정
- 0.5의 확률로 뽑힌 개수를 20번 수행하여 출력



```
import numpy as np
n, p = 10, 0.3

bimodal_sample = np.random.binomial(n, p, 20)
print("이항 분포 샘플: {}".format(bimodal_sample))
```

이항 분포 샘플: [2 1 3 2 4 3 2 1 3 3 3 3 5 6 5 2 7 1 3 4]

- 이항 분포
- 시행 10번 $p=0.3$ 로 설정
- 0.3의 확률로 뽑힌 개수를 20번 수행하여 출력



```
import numpy as np

uniform_sample = np.random.randint(0,10,20)
print("이산 균등 분포 샘플: {}".format(uniform_sample))
```

이산 균등 분포 샘플: [7 4 5 4 8 1 3 1 6 8 9 1 0 7 1 4 4 9 9 2]

- 이산 균등 분포
- randint()를 이용하여 랜덤으로(모두 같은 확률로) 샘플링



연속 변수 X 가 a 보다 크고 b 보다 작은 범위에서 정의된다면
모두 같은 확률 값을 갖는 확률 분포

- $P(X = x) = \frac{1}{b-a}$
- 기대값 $\frac{a+b}{2}$
- 분산 $\frac{(b-a)^2}{12}$



어떤 사건이 처음 발생할 때까지의 경과 시간에 대한 연속 확률 분포

- $P(X = x) = \lambda e^{-\lambda x}$
 - λ 는 단위 시간 동안 평균 이벤트 발생 횟수
- 사건이 일정한 평균 비율로 발생한다고 가정
- 사건이 발생한 시간 간격을 모델링하는데 사용



평균값에서 가장 높은 확률값을 가지며
그 지점을 중심으로 대칭을 이룬 확률 분포

- $P(X = x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2}}$
- 자연계에서 많은 변수들이 따르는 분포
- 중심극한정리에 의해 독립적인 랜덤 변수들이 합을 따름



```
import numpy as np
```

```
uniform_sample = np.random.uniform(0,10,20)
```

```
[8.43056234, 6.76236261, 1.83144344, 1.27609253, 6.27857984,  
 5.84250982, 2.82498284, 4.79871319, 8.07585953, 1.78467954,  
 6.32170234, 7.09163585, 5.42811678, 2.93592529, 2.08821724,  
 0.27477224, 3.4177471 , 2.19335609, 4.82969174, 4.41063123]
```

- 연속 균등 분포
- random.uniform()
 - 지정된 범위에서 균일한 분포를 가지는 난수를 생성



```
# beta = 1/lambda  
beta = 1  
exp_sample = np.random.exponential(beta, 20)
```

```
[1.17344827, 0.61064236, 0.58848131, 1.75938518, 0.36647312,  
0.30549056, 0.05164886, 0.06601667, 2.00596693, 0.59239035,  
0.38043764, 0.00805003, 0.61846678, 1.32281079, 0.86826902,  
0.17589109, 2.23817701, 0.28699922, 0.463474 , 2.37824995]
```

- 지수 분포
- random.exponential()
 - 지수분포를 따르는 난수 생성 함수



```
mu, sigma = 0, 0.1  
normal_sample = np.random.normal(mu, sigma, 20)
```

```
[ 0.07511448, -0.09333431,  0.07912994,  0.11550446, -0.11794959,  
 0.01365663,  0.17581952,  0.08071331,  0.18074644,  0.08877352,  
-0.13509446, -0.08328683,  0.05512134, -0.056933  ,  0.02541844,  
-0.15964837,  0.08310834,  0.07835469,  0.05698007, -0.06349036]
```

- 정규 분포
- random.normal()
 - 정규분포를 따르는 난수 생성 함수



어떤 사건 A가 일어났다는 조건 하에서 다른 조건 B가 일어날 확률을 나타내는 개념

- $P(B|A)$
- 사건들이 서로 **의존적**일 때 매우 유용
- **두 가지 사건에 대한 관계**를 확률에 적용



```
import numpy as np
A = [1, 2, 3, 4]
B = [2, 4, 6, 8, 10, 12]
U = set(A+B)
prob_A = len(A)/len(U)
prob_B = len(B)/len(U)

prob_A_and_B = len(np.intersect1d(A,B))/len(U)

conditional_P_B_A = prob_A_and_B/prob_A
```

- **intersect1d(A,B)**
 - 사건 A와 B의 교집합을 구하는 함수



조건부 확률을 계산하는 데 사용되는 중요한 수학적 정리

- $P(A|B) = \frac{P(B|A) P(A)}{P(B)}$
- $P(A|B)$: 사건 B가 주어졌을 때 사건 A가 발생할 확률 (사후 확률, posterior)
- $P(B|A)$: 사건 A가 주어졌을 때 사건 B가 발생할 확률 (우도, likelihood)
- $P(A)$: 사건 A가 발생할 확률 (사전 확률, prior probability)
- $P(B)$: 사건 B가 발생할 확률 (정규화 상수, evidence)

9. 통계 분석



데이터를 수집, 정리, 분석하여 **유의미한 패턴과 결론을 도출**하는 과정

- 머신러닝에서 사용되는 데이터 또한 통계 분석을 통하여 특징을 파악할 수 있음
- 데이터가 변형됨에 따라 비교할 수 있는 수학적 파라미터 제시

데이터를 요약하는 정보

- 확률 변수 또는 데이터값의 합을 전체 개수로 나눈 값
- N개의 데이터 평균 $data = \sum_{i=1}^N data_i$
- 이산 확률변수 평균 $E[X] = \sum_{i=1}^N x_i p_i$
- 연속 확률변수 평균 $E[X] = \int P(x) dx$



데이터가 얼마나 퍼져있는지를 나타냄

- 편차(한 데이터와 전체 데이터 평균의 차) 제곱 합의 평균

- N개의 데이터 분산 $var = \frac{1}{N} \sum_{i=1}^N (data_i - \overline{data})^2$

- **표준 편차 = 분산의 제곱근 값**

- $\sigma = \sqrt{var}$

- 확률 변수 표준편차

- $\sigma = \sqrt{E[(X - E[X])^2]}$



```
import numpy as np
X = np.random.normal(1, 10, 1000)

mean_X = np.mean(X)
std_X = np.std(X)
```

- **mean()**
 - 평균을 반환해주는 함수
- **std()**
 - 표준편차를 반환해주는 함수



두 변수 간의 선형 관계와 강도, 방향을 측정하는 통계적 지표

- $\rho_{X,Y} = \frac{cov(X,Y)}{\sigma_X \sigma_Y}$
- -1에서 1 사이의 값을 가짐
 - -1 : 두 변수 간의 음의 선형 관계
 - 0 : 두 변수 간의 선형 관계 없음
 - 1 : 두 변수 간의 양의 선형 관계



```
import numpy as np
X = np.random.normal(1,10,1000)
Y = np.random.normal(3,5,1000)

corr_XY = np.corrcoef(X,Y)
```

- **corrcoef()**
 - 입력된 두 변수 간의 상관계수를 구해주는 함수